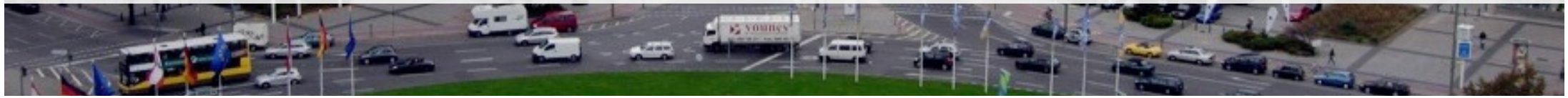


MODUL KÜNSTLICHE INTELLIGENZ: GRUNDLAGEN UND ANWENDUNGEN

PROBLEMLÖSEN DURCH SUCHE

Suchproblem | Problemraum und Suchbaum | uninformierte Suche | informierte Suche

Dr.-Ing. Stefan Fricke | Fachgebiet *Service-centric Networking* | TU Berlin



Problemlösen durch systematische Suche

Gegeben: 2 leere Krüge mit unterschiedlichem Fassungsvermögen, Wasserquelle
Aufgabe: Füllen Sie, mit minimaler Zahl von Aktionen, exakt 2 Liter in einen Krug!
Aktionen: Füllen, Umfüllen, Leeren



Umgebung des Agenten: statisch, deterministisch, zugänglich

Problemlösen durch Suche:

- Input: Suchproblem & Startzustand
  - Generierung neuer Zustände ...
  - ... durch systematische Anwendung von Aktionen ...
  - bis Ziel erreicht,
- alles im Kopf des Agenten

Ein Suchproblem besteht aus Startzustand, Zieltest, Operatoren und Aktionskosten

S: Menge aller erreichbaren Zustände
A: Menge aller Aktionen

Startzustand	$s_0 \in S$
Zieltest	$zt : S \rightarrow bool$
Aktionen (Operatoren)	$a_i \in$ A mit a_i : S → S
Aktionskosten	$k : a_i \in \mathbb{R} \quad \forall a_i \in A$

Aktionskosten üblicherweise positiv!

Suchproblembeschreibung für Krüge umfüllen:

s_0 = { (4,0), (3,0) }

$zt(s) = IF\ (x,2) \in s\ THEN\ true\ ELSE\ false$

Füllen(k,s) = s': IF $(k,x) \in s$
THEN $s' = s \setminus \{(k,x)\} \cup \{(k,k)\}$

Leeren(k,s) und Umfüllen(k,l,s) analog

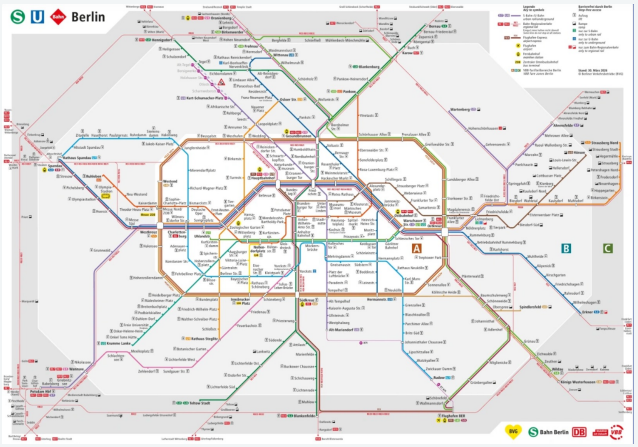
uniforme Aktionskosten ($k = 1$ für jede Aktion)

Lösung: Sequenz von Zuständen $(s_0, s_1, \dots, s_k)$, wobei $zt(s_k) = true$ und $\forall i \in 0, \dots, k-1\ \exists a_i \in A$ sodass $a_i(s_i) = s_{i+1}$

Optimale Lösung: Lösung mit minimalen Kosten

Beispiele für Suchprobleme

Routenplanung:
- Zustand: Ort
- Operator: Nachbarort anfahren
- Aktionskosten: Entfernungen
- Ziel: Ort

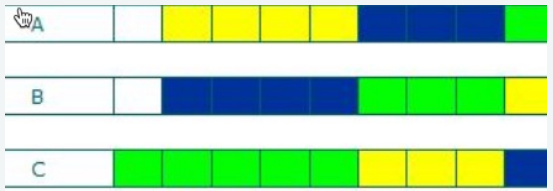


Bildquelle: bvg.de

Sudoku:
- Zustand: Konfiguration
- Operator: Zahl eintragen
- Ziel: vollständig & konsistent
- Aktionskosten uniform (egal, weil keine Optimierung)

1		2						
		3						
								4
	4			5				
	6			7				
							2	
	8							
						8		

Job Shop Scheduling:
- Zustand: Maschinenbelegung und offene Jobs
- Operator: Job einplanen
- Aktionskosten: Erhöhung der Gesamtdauer
- Ziel: keine offenen Jobs



Türme von Hanoi ..



A|O|T
Agententechnologien in betrieblichen Anwendungen und der Telekommunikation

2.2 Vom Problemraum zum Suchbaum

- Problemraum
- Suchbaum
- Suchbaumalgorithmus

© Dr.-Ing. Stefan Fricke

Zustände und Aktionen beim Krüge umfüllen bilden einen Graphen

Suchproblembeschreibung:

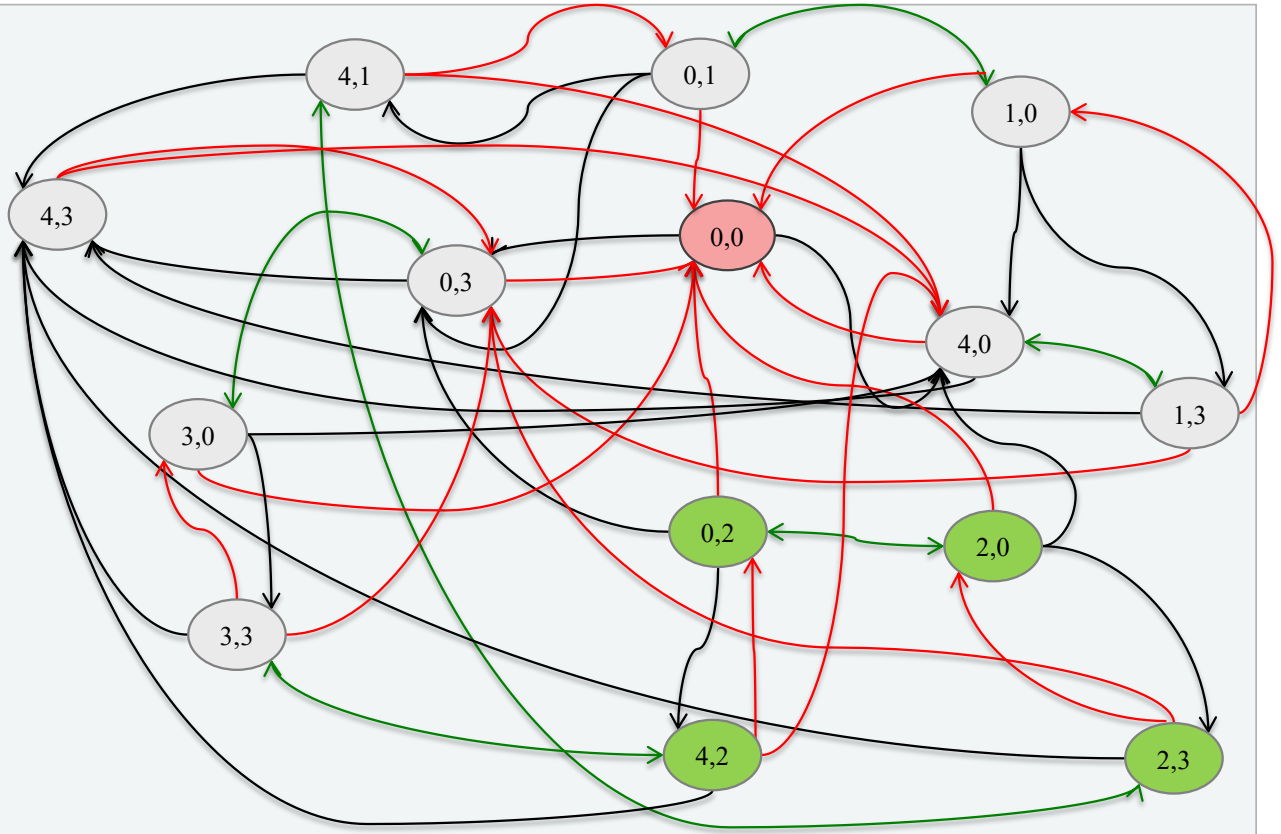
$s_0 = (0,0)$

$zt((x,y)) = (x = 2 \ OR \ y = 2)$

Aktionen: Füllen, Leeren, Umfüllen

uniforme Aktionskosten

Suche = systematische Exploration des Problemraums, beginnend in s_o

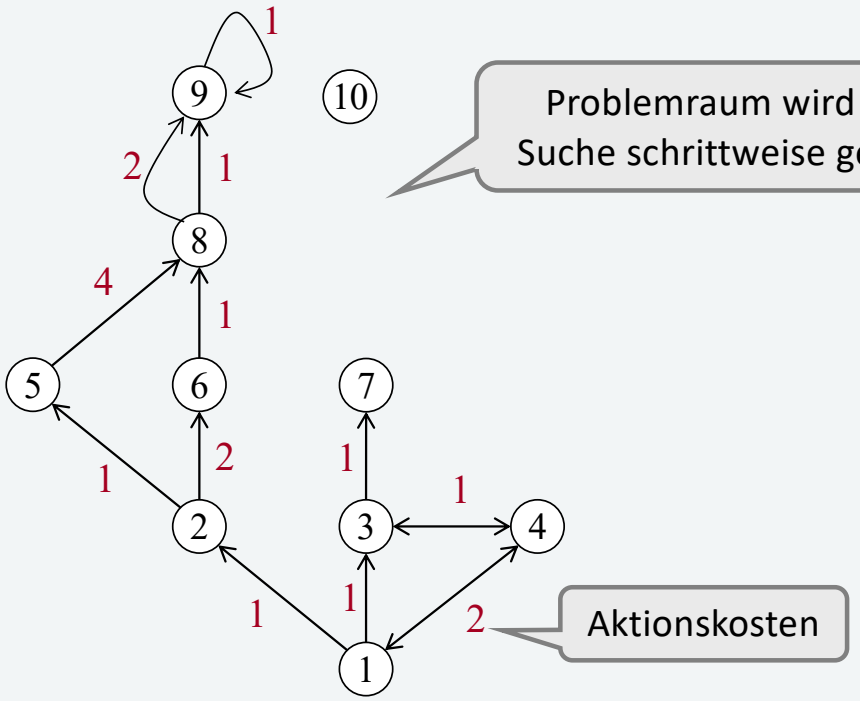


Zustand | Startzustand s_0 | Zielzustand | Füllen | Leeren | Umfüllen

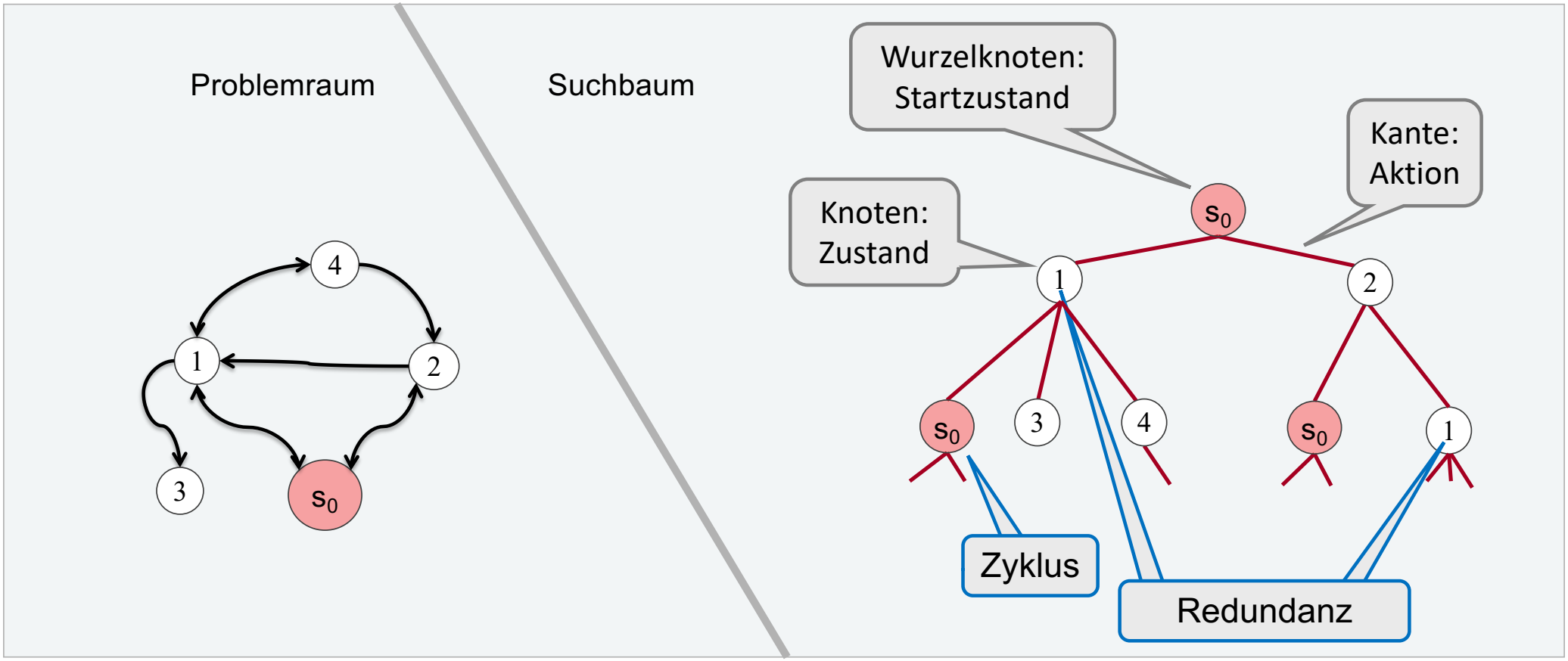
Der Problemraum ist ein gerichteter, gewichteter Multigraph

Herausforderungen bei der Suche:

- Zyklen
- Sackgassen
- Redundanzen
- Unendlich große Problemräume
- Unlösbarkeit



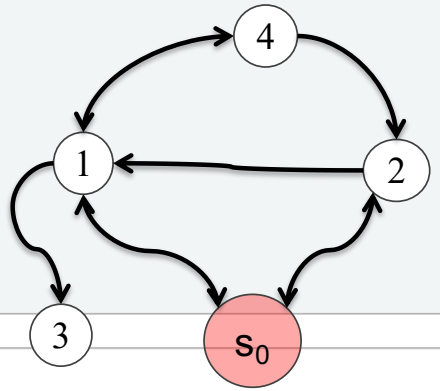
Jeder Problemraum ist als Suchbaum darstellbar

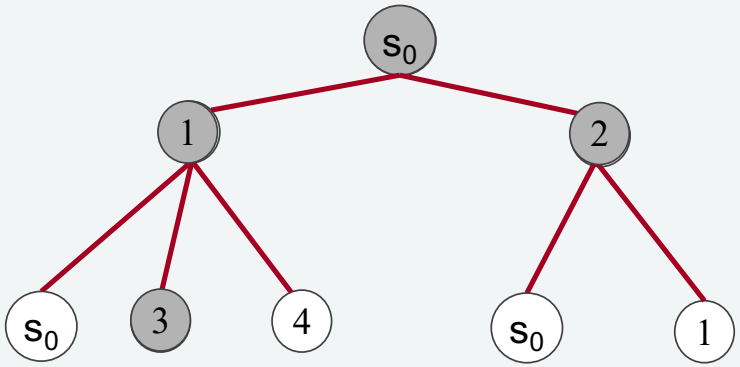


Der Suchbaum wird durch Expansion mittels einer Strategie schrittweise generiert

(Winston 1993)

1. Form a one-node tree consisting of the initial state as root, color root white.
2. Until there is no white leaf
    a) Select next white leaf as current leaf according to a strategy.
    b) Test: If the current leaf represents the goal state, return success.
    c) Color current leaf as grey.
    d) Expand current leaf and color its child nodes white.
3. Return failure.





A|O|T
Agententechnologien in betrieblichen Anwendungen und der Telekommunikation

2.3 Uninformierte Suchverfahren

- Tiefensuche
- Breitensuche
- Redundanzvermeidung

© Dr.-Ing. Stefan Fricke

Tiefensuche mit Redundanzvermeidung verwendet Stack von Pfaden

last in, first out

1 PUSH((s_0), $Stack$), $Generated = \{s_0\}$
2 Until $Stack$ nicht leer
3 $(s_0, \dots, s_k)$ = POP($Stack$)
4 IF $zt(s_k)$ THEN $return$ $(s_0, \dots, s_k)$
5 ELSE
6 NF = expandiere(s_k)
7 $\forall x \in NF$ mit $x \notin Generated$
8 PUSH($(s_0, \dots, s_k, x)$, $Stack$)
9 $Generated = Generated \cup NF$
10 Return Failure

strategy = maximale Tiefe

1. Form a one-node tree consisting of the initial state as root, color root white.
2. Until there is no white leaf
   a) Select next white leaf as current leaf according to a strategy.
   b) Test: If the current leaf represents the goal state, return success.
   c) Color current leaf as grey.
   d) Expand current leaf and color its child nodes white.
3. Return failure.

Tiefensuche und Breitensuche im Vergleich

Strategie = First In, First Out

1	PUSH((s_0), $Stack$), $Generated = \{s_0\}$	ENQUEUE((s_0), Q), $Generated = \{s_0\}$
2	Until $Stack$ nicht leer	Until Q nicht leer
3	$(s_0, \dots, s_k)$ = POP($Stack$)	$(s_0, \dots, s_k)$ = DEQUEUE(Q)
4	IF $zt(s_k)$ THEN $return$ $(s_0, \dots, s_k)$	IF $zt(s_k)$ THEN $return$ $(s_0, \dots, s_k)$
5	ELSE	ELSE
6	NF = expandiere(s_k)	NF = expandiere(s_k)
7	$\forall x \in NF$ mit $x \notin Generated$	$\forall x \in NF$ mit $x \notin Generated$
8	PUSH($(s_0, \dots, s_k, x)$, $Stack$)	ENQUEUE($(s_0, \dots, s_k, x)$, Q)
9	$Generated = Generated \cup NF$	$Generated = Generated \cup NF$
10	Return Failure	Return Failure

2.4 Eigenschaften von Breiten- und Tiefensuche

- Vollständigkeit
- Optimalität
- Laufzeit- und Speicheranforderungen

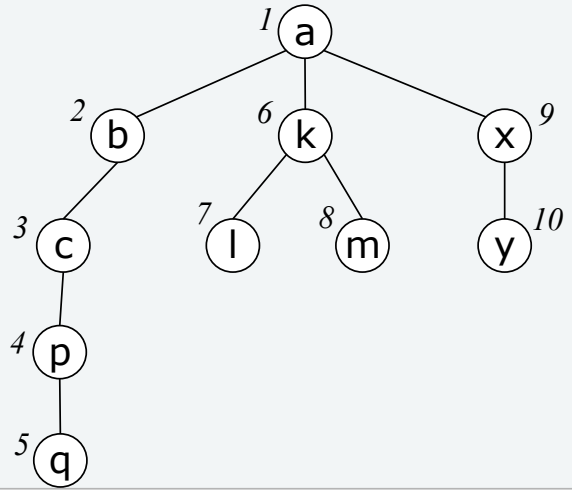
© Dr.-Ing. Stefan Fricke

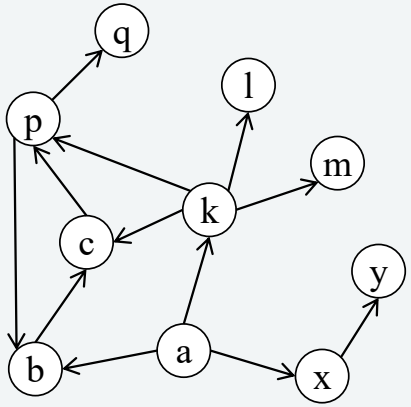
Tiefen- und Breitensuche im Vergleich

Tiefensuche

Strategie: Expandiere Blatt mit **maximaler Tiefe**, vermeide Redundanz.

Speichere **Pfade** in **Stack** (LIFO)

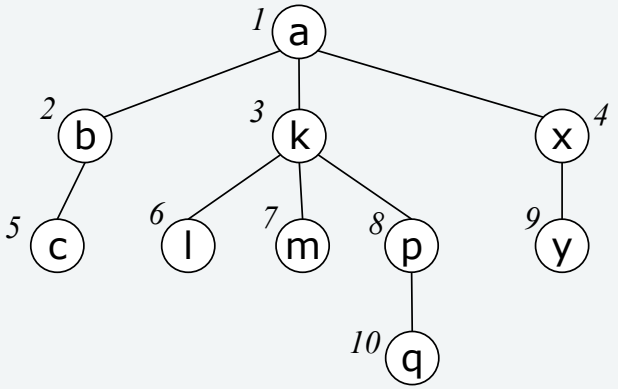




Breitensuche

Strategie: Expandiere Blatt mit **minimaler Tiefe**, vermeide Redundanz.

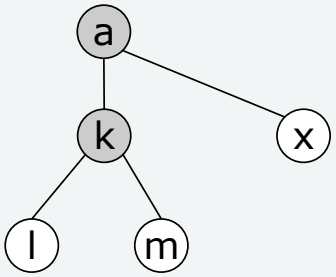
Speichere **Pfade** in **Queue** (FIFO)



Breitensuche ist speicheraufwändiger als Tiefensuche

Tiefensuche

Zustand nach Expandierung von k

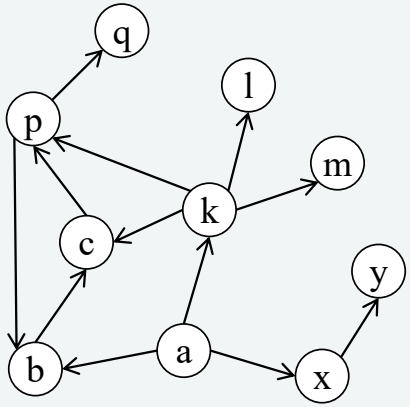


Stack:

(a,k,l)

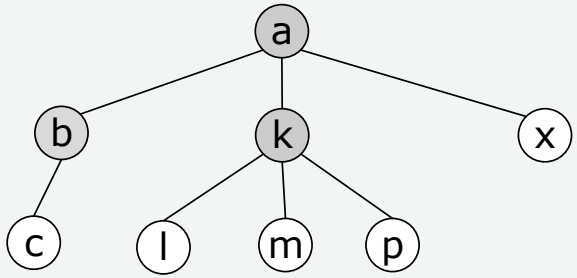
(a,k,m)

(a,x)



Breitensuche

Zustand nach Expandierung von k



Queue:

(a,x)

(a,b,c)

(a,k,i)

(a,k,m)

(a,k,p)

4 Kriterien zur Leistungsmessung von Suchalgorithmen

Vollständigkeit: irgendeine Lösung garantiert?

Optimalität: optimale Lösung garantiert?

= minimale „Pfadkosten“ (akkumulierte Aktionskosten)

Zeitkomplexität: im Worst Case ...

Speicherkomplexität: im Worst Case ...

Zeit- und Speicherkomplexität werden anhand der generierten Knoten gemessen

Zeitkomplexität:

Anzahl erzeugter Knoten

Speicherkomplexität:

Maximale Zahl der Knoten im Speicher

Faktoren zur Komplexitätsbestimmung:

b: Verzweigungsgrad (**b**ranching factor)

o: Tiefe der **o**ptimalen Lösung (depth)

m: **m**aximale Tiefe des Suchbaums

1. Form a one-node tree consisting of the initial state as root, color root as white.
2. Until there is no white leaf
   a) Select next white leaf as current leaf according to a **strategy**
   b) Test: If the current leaf represents the goal state, return success.
   c) Color current leaf as grey
   d) Expand current leaf and color its child nodes white
3. Return failure.

Tiefensuche ist speichereffizient, Breitensuche garantiert bei uniformen Aktionskosten Optimalität

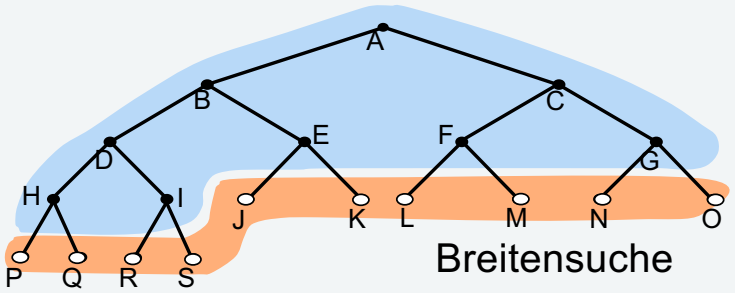
Kriterium	Breitensuche	Tiefensuche
Vollständig	Ja	Ja*
Optimal	Ja	Nein
Laufzeit	$O(b^o)$	$O(b^m)$
Speicher	$O(b^o)$	$b \cdot m$

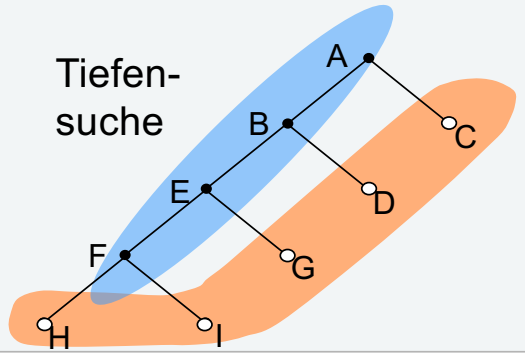
* in endlichen Bäumen

o: Tiefe optimaler Lösung

m: maximale Baumtiefe

b: Verzweigungsgrad





A|O|T
Agententechnologien in betrieblichen Anwendungen und der Telekommunikation

2.5 Iterative Tiefensuche

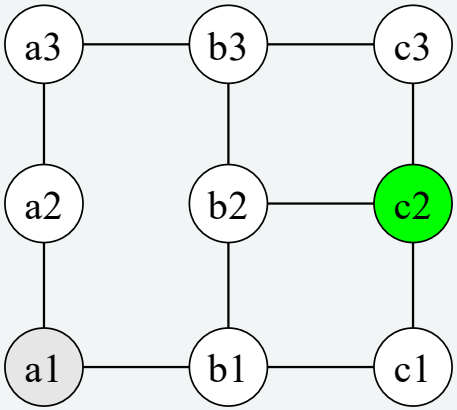
Beschränkte Tiefensuche

Iterative Tiefensuche

© Dr.-Ing. Stefan Fricke

Beschränkte Tiefensuche findet Lösung innerhalb eines begrenzten Suchhorizonts

```
1  DLS( path,  stack,  depth )
2        IF zt( last(path) ) THEN return path
3        stack = expand( path )
4        Until Stack nicht leer
5              path = POP( Stack )
6              IF length(path) < depth
7                    THEN DLS( path,  stack,  depth )
```



depth < 3 findet keine Lösung

depth = 3 findet (a1,b1,b2,c2)*

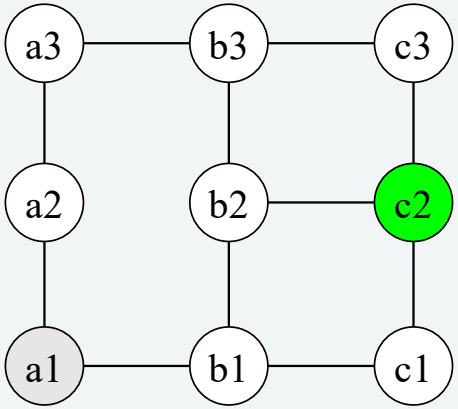
depth ≥ 5 findet (a1,a2,a3,b3,b2,c2)*

DLS = Depth Limited Search

* bei lexikographischer Reihenfolge

Iterative Tiefensuche ähnelt der Breitensuche

```
1    IDS( (s_0) )
2        FOR depth = 0 to ∞
3            (path, continue) = DLS( (s_0), [ ], depth )
4            IF path ≠ ( ) THEN return path
5            ELSE IF not continue THEN return not found
```



vollständiger Pseudocode:
en.wikipedia.org/wiki/Iterative_deepening_depth-first_search

Untersuchte Zustände:

Iteration *0*: a1

Iteration *1*: a1, a2, b1

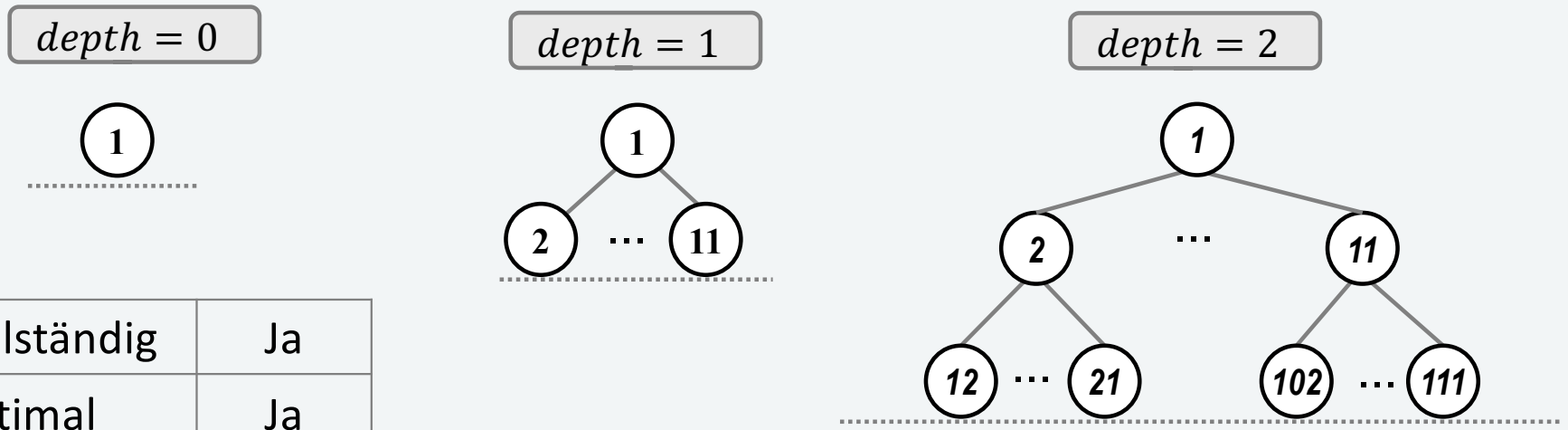
Iteration *2*: a1, a2, a3, b1, b2, c1

Iteration *3*: a1, a2, a3, b3, b1, b2, c2

IDS = Iterative Deepening Search; DLS = Depth Limited Search

Iterative Tiefensuche vereinigt die besten Eigenschaften von Breiten- und Tiefensuche

Beispiel mit Verzweigungsgrad 10:



vollständig	Ja
optimal	Ja
Zeit	O (b^o)
Speicher	O ($b \cdot d$)

„relativ geringer“ Overhead

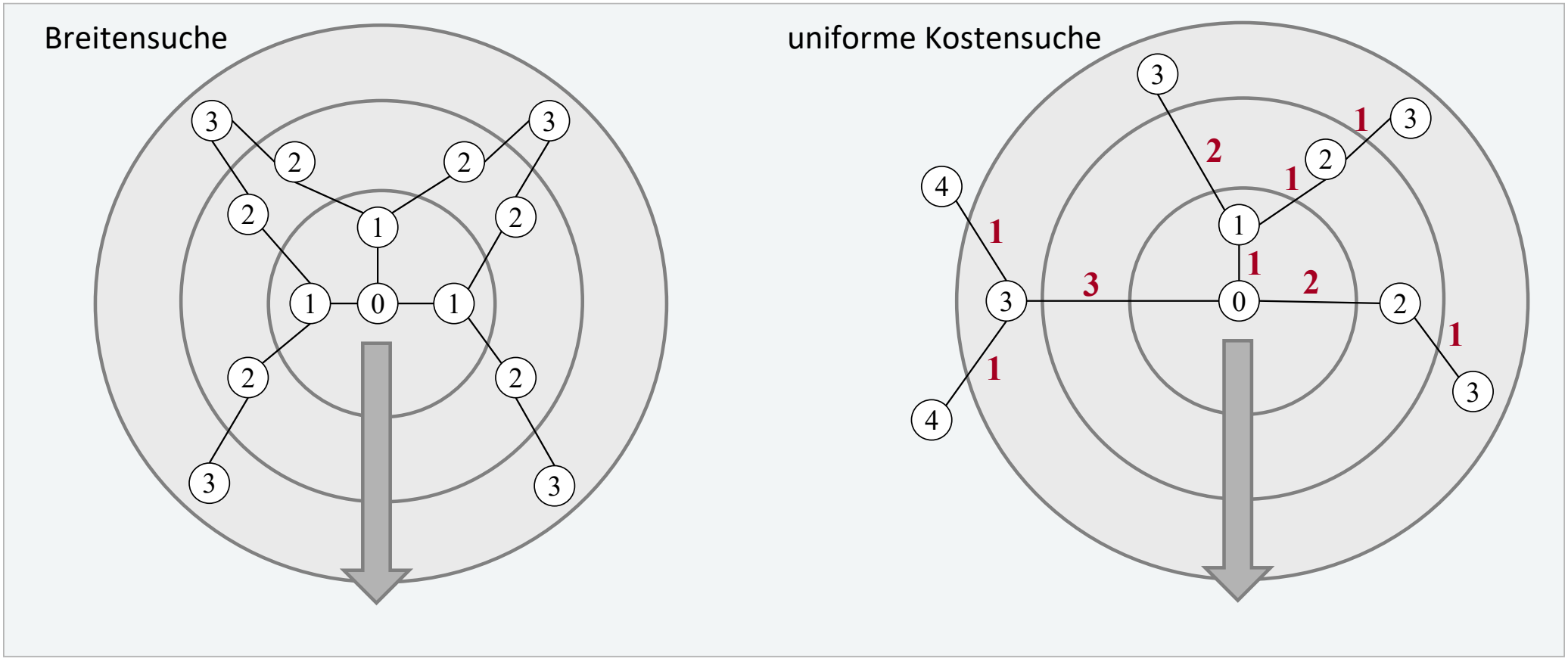
2.6 Uniforme Kostensuche

... berücksichtigt nicht uniforme Aktionskosten,

... verwendet dynamische Programmierung zur Redundanzvermeidung

© Dr.-Ing. Stefan Fricke

Von uniformen zu nichtuniformen Kosten

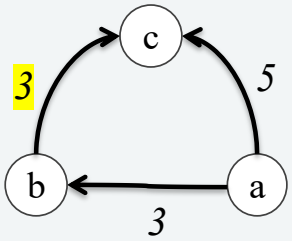


(2) In Zuständen stehen die akkumulierten Aktionskosten (Pfadkosten).

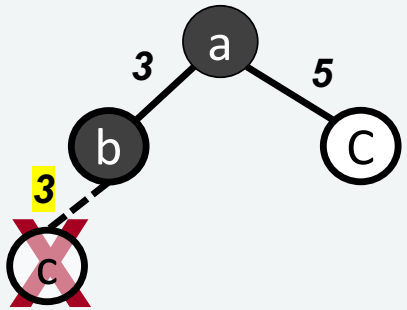
Untersuchungs-reihenfolge

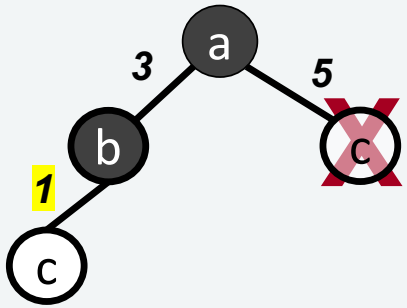
3 Aktionskosten in roter Schrift.

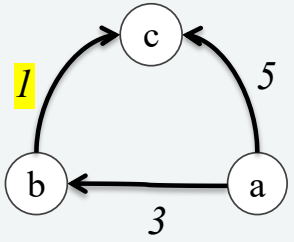
Dynamische Programmierung ist generalisierte Redundanzvermeidung



Situationen beim Expandieren von B:







generierte Zustände werden mit Pfadkosten gespeichert – nur der billigste überlebt:

x	g(x)
a	0
b	3
c	5

x	g(x)
a	0
b	3
c	5

x	g(x)
a	0
b	3
c	4

Uniforme Kostensuche verwendet nach Pfadkosten sortierte Liste von Pfaden

1. Form a one-node tree consisting of the initial state as root, color root as white.
   Initialize DP-Table with g(root) = 0.
2. Until there is no white leaf
   a) Select next white leaf as current leaf according to min(f=g)
   b) If goal test on current leaf succeeds, return path to current leaf.
   c) Color current leaf as grey
   d) Expand current leaf, apply dynamic programming, and color the remaining child nodes white.
3. Return failure.

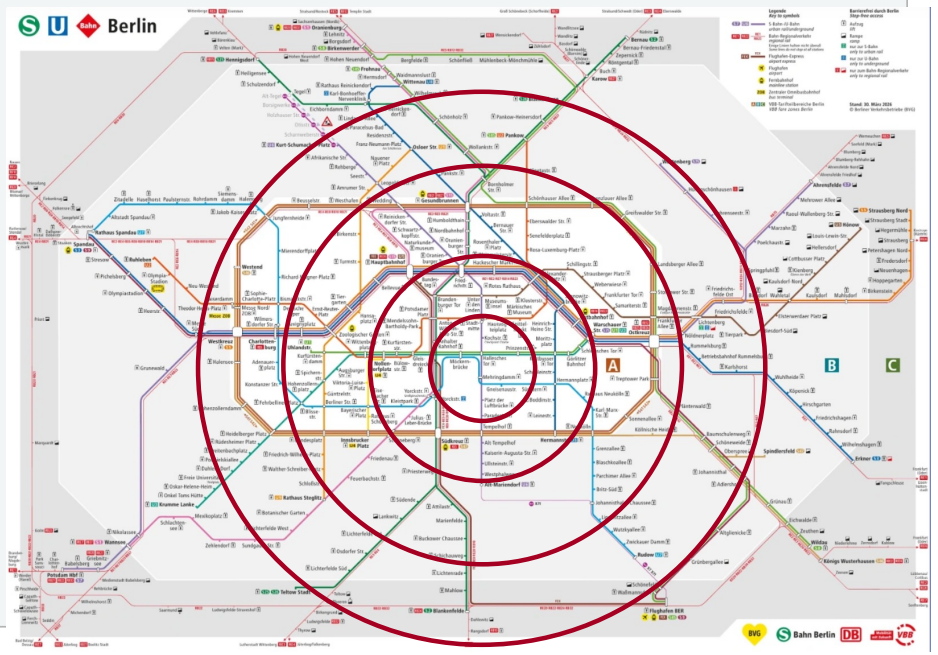
Uniforme Kostensuche garantiert Optimalität, arbeitet aber nicht zielgerichtet

Kriterium	Breitensuche	uniforme Kostensuche
Vollständig	ja*	ja*
Optimal	ja	ja
Laufzeit	$O(b^o)$	$O(b^m)$
Speicher	$O(b^o)$	$O(b^m)$

o: Tiefe optimaler Lösung

m: maximale Baumtiefe

b: Verzweigungsgrad



* bei endlichem Verzweigungsgrad

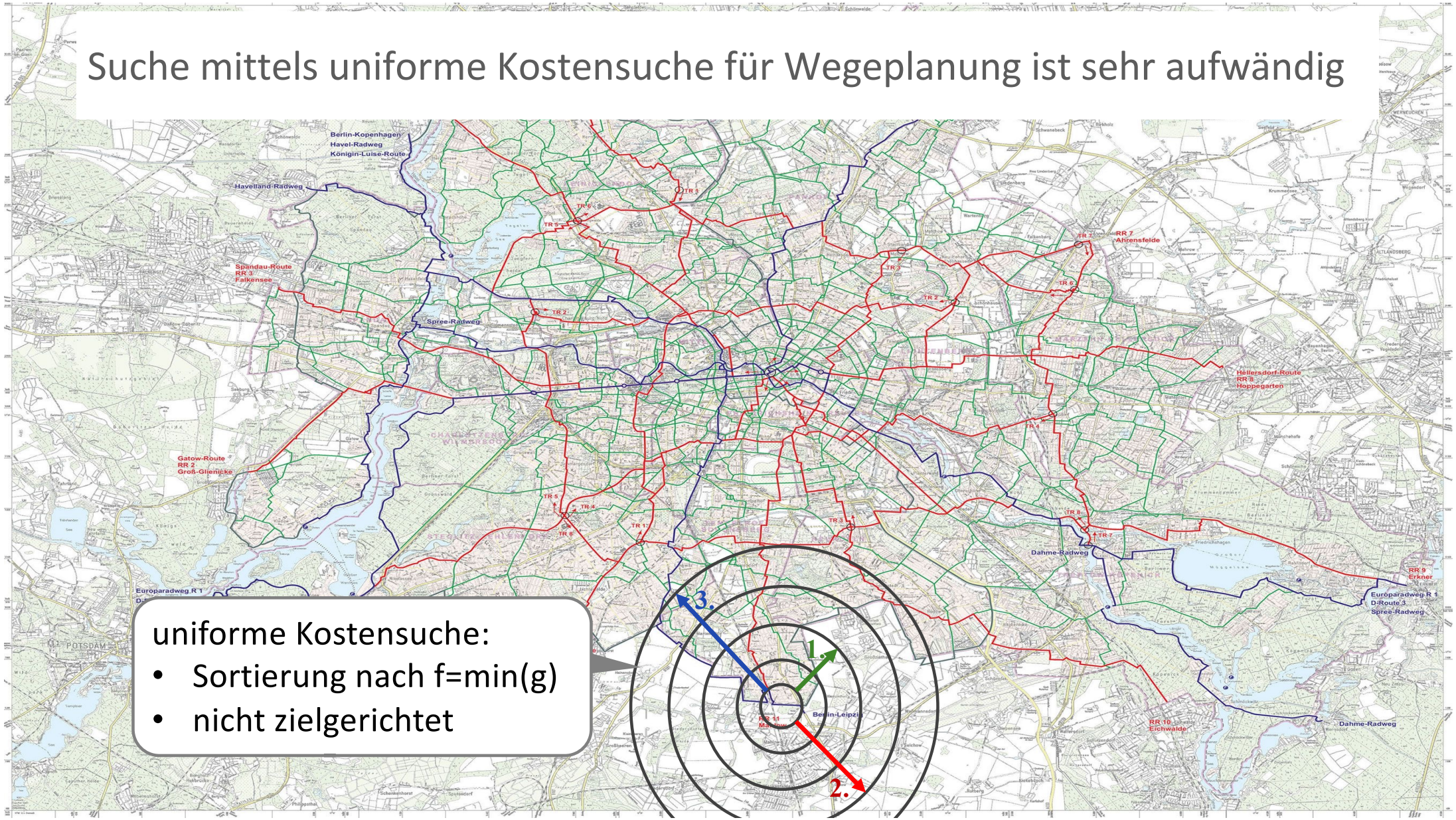
A|O|T
Agententechnologien in betrieblichen Anwendungen und der Telekommunikation

2.7 Informierte Suche

- Heuristiken
- A*

© Dr.-Ing. Stefan Fricke

Suche mittels uniforme Kostensuche für Wegeplanung ist sehr aufwändig



uniforme Kostensuche:

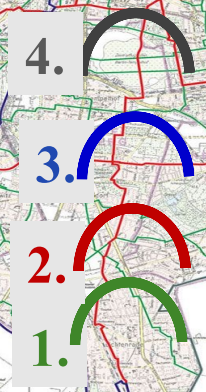
- Sortierung nach f=min(g)
- nicht zielgerichtet

Heuristische Suche sortiert offene Pfade nach Restkostenschätzwerten

heuristische Funktion

$$h\colon S \to \mathbb{R}^{0+}$$

definiert für jeden Zustand eine Schätzung der Kosten zum Ziel

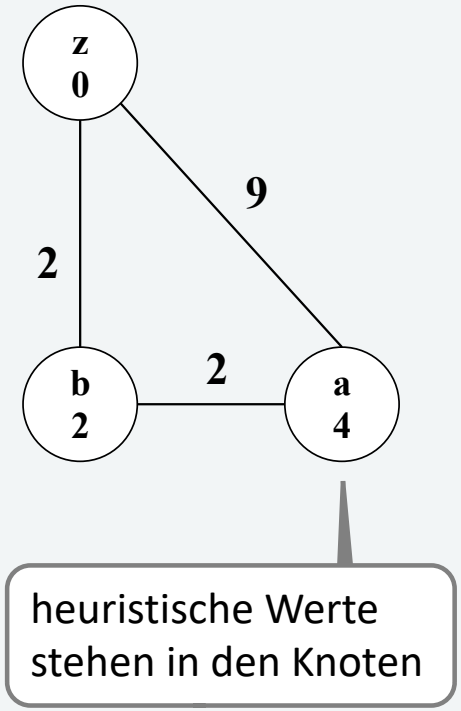


gute Heuristik für Path Planning: Luftlinienentfernung zum Ziel

Heuristische Suche:

- Sortierung nach f=min(h)
- zielgerichtet

Heuristische Suche garantiert auch mit perfekt informierter Heuristik keine Optimalität



Lege Pfad (s_0) in $Open$, lege Zustand s_0 in $Generated$

Until $Open$ nicht leer

Entferne Pfad $(s_0, \dots, s_k)$ aus $Open$ gemäß Strategie f = min(h)

IF $zt(s_k) = true$

THEN $return\ (s_0, \dots, s_k)$

ELSE Für jeden Nachfolger x von $s_k \notin Generated$

lege x in $Generated$

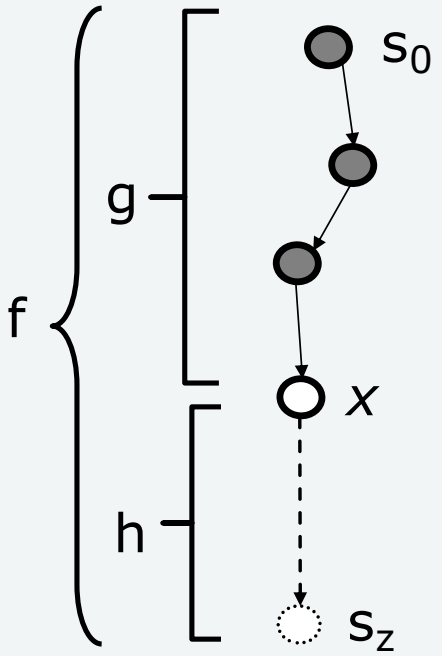
lege Pfad $(s_0, \dots, s_k, x)$ in $Open$

Return Failure

A* verwendet Pfadkosten und Heuristik

Sortierung der offenen Pfade nach f = min(g + h)
- g = Pfadkosten (akkumulierte Kosten)
- h = heuristische Funktion (Vorwissen)

1. Form a one-node tree consisting of the initial state s_0 as root, color root as white. Initialize DP-Table with $g(s_0) = 0$.
2. Until there is no white leaf
   a) Select next white leaf *x* according to min(f = g+h)
   b) If goal test on *x* succeeds, return path to current leaf
   c) Color current leaf as grey
   d) Expand current leaf, apply dynamic programming, and color the remaining child nodes white
3. Return failure.



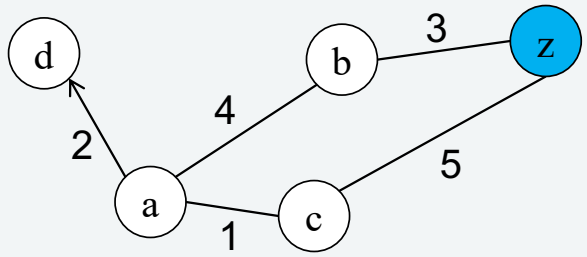
P.E. Hart, N.J. Nilsson, B. Raphael: A Formal Basis for the Heuristic Determination of Minimum Cost Paths. IEEE Transactions on Systems Science and Cybernetics SSC4 (2), 1968, S. 100–107.

2.8 Eigenschaften von Heuristiken

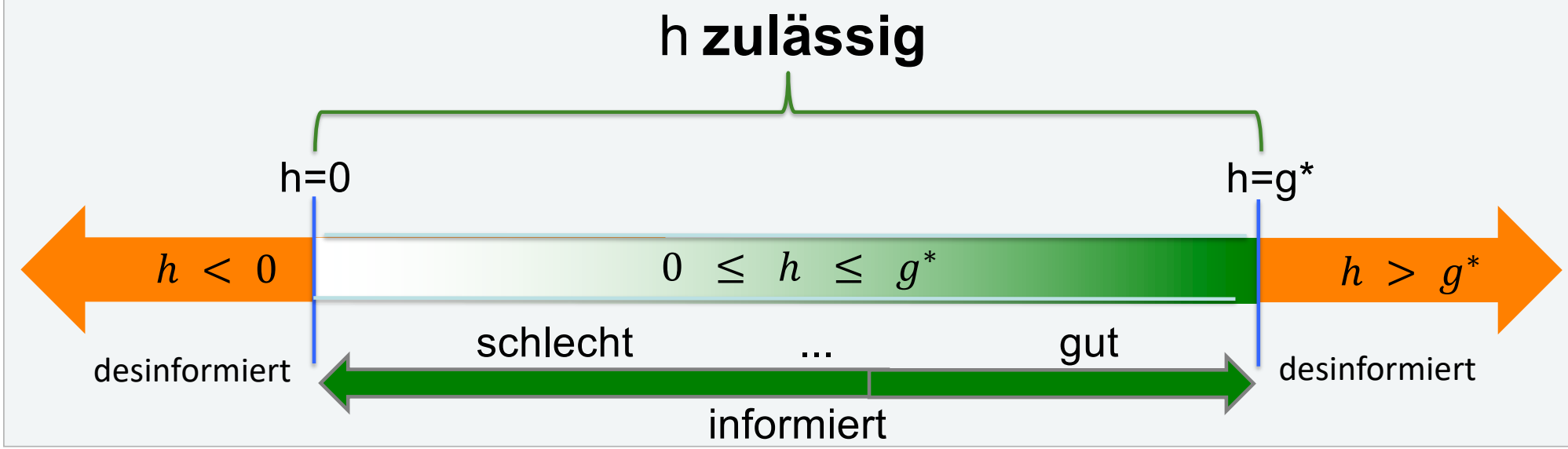
- Zulässigkeit
- Monotonie
- Dominanz

© Dr.-Ing. Stefan Fricke

Eine zulässige Heuristik überschätzt nicht die tatsächlichen Kosten



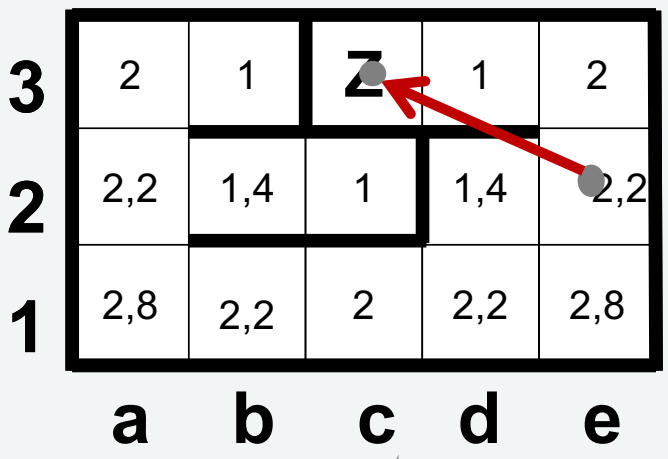
$0 \le h(a) \le 6$, $0 \le h(b) \le 3$
$0 \le h(c) \le 5$ $0 \le h(d) \le \infty$
$h(z) = 0$



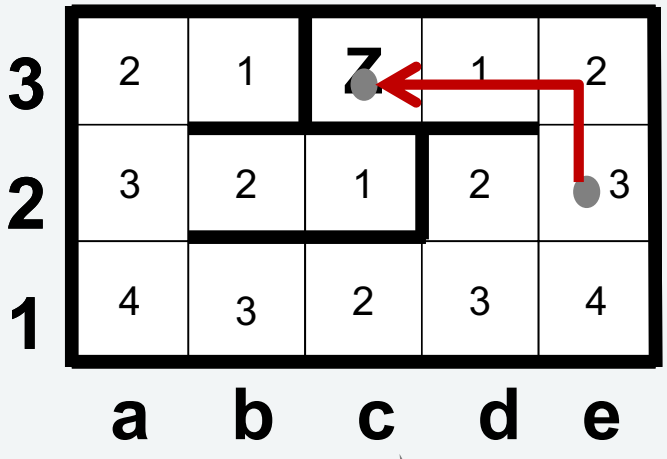
g^* : minimale Pfadkosten zum Ziel

realistische Annahme: Nichtnegative Aktionskosten

Manhattandistanz dominiert Luftlinienentfernung



Luftlinienentfernung h_{LL} ist eine gute Heuristik



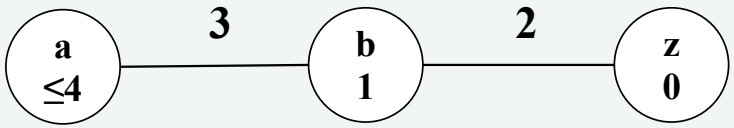
Manhattandistanz h_{MM} ist eine bessere Heuristik

Dominanz: h_{MD} zulässig und $h_{MD} \geq h_{LL}$

Monotonie ist eine noch stärkere Eigenschaft als Zulässigkeit

h ist monoton, wenn $\forall i\colon\ h(\,x_i\,) \leq\ h(\,x_{i+1}\,)\ +\ c(\,x_i, x_{i+1}\,)$

(h zulässig *und* überschätzt die Aktionskosten c nicht)

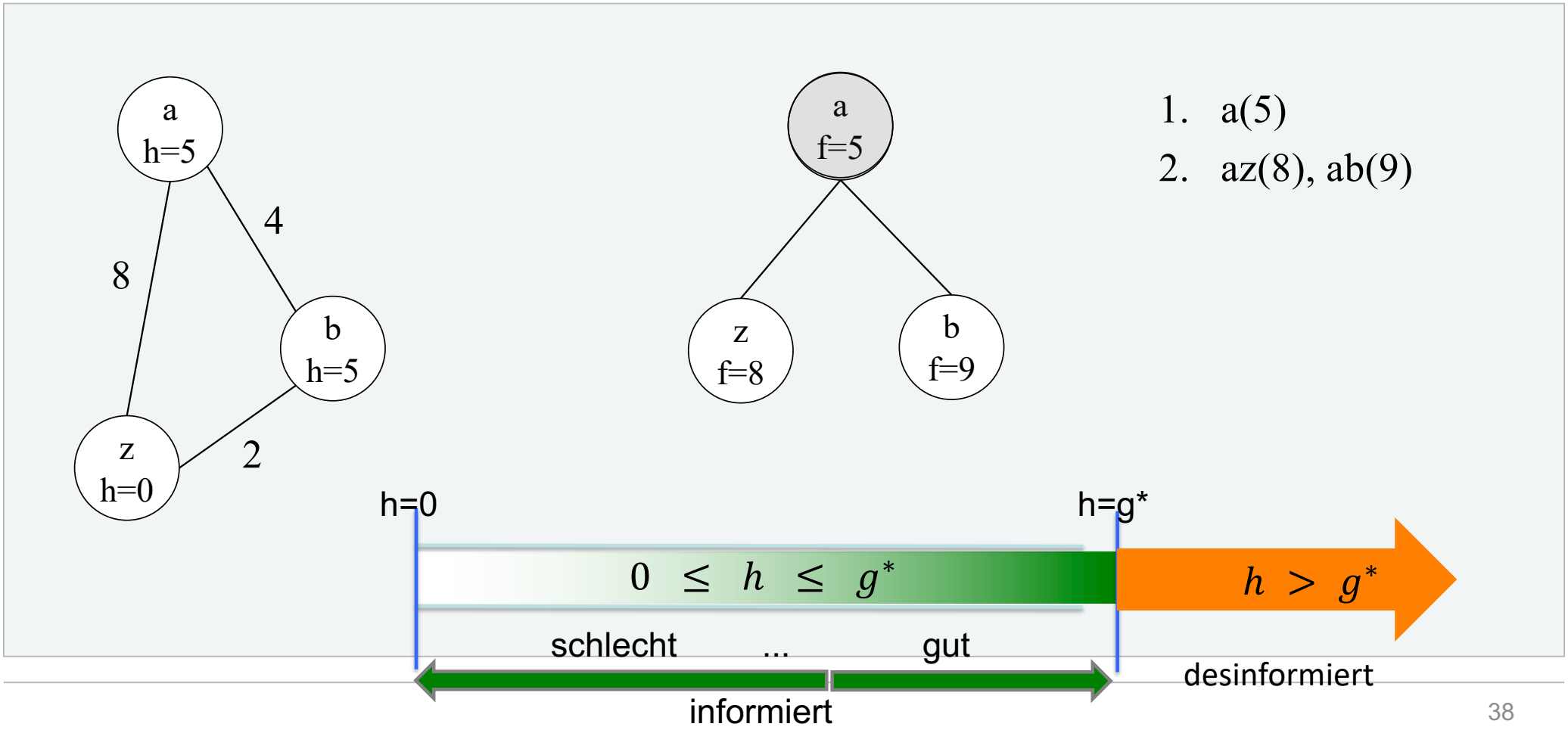


Schätzfehler (Optimismusfaktor) zum Ziel hin monoton fallend

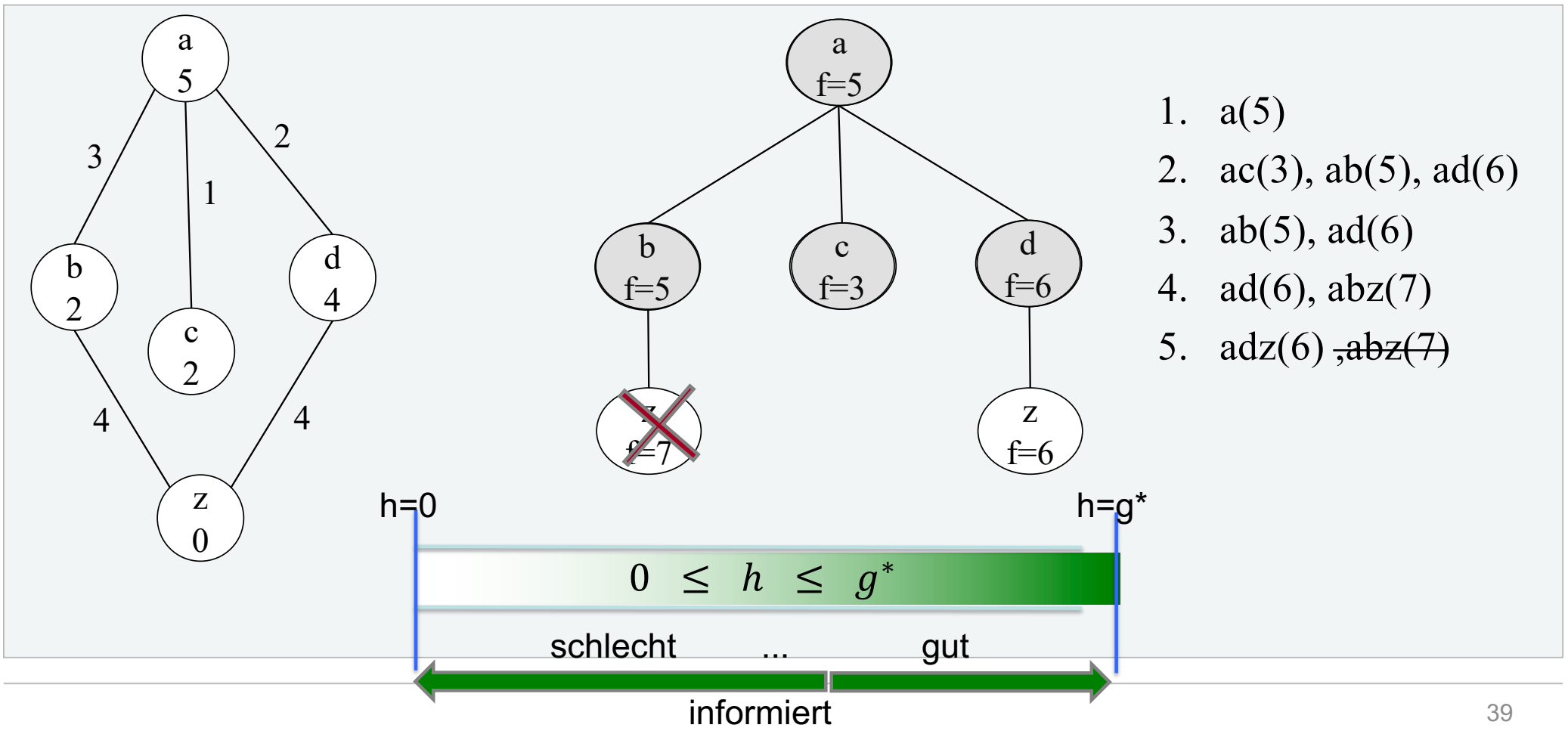
2.9 Eigenschaften von A*

© Dr.-Ing. Stefan Fricke

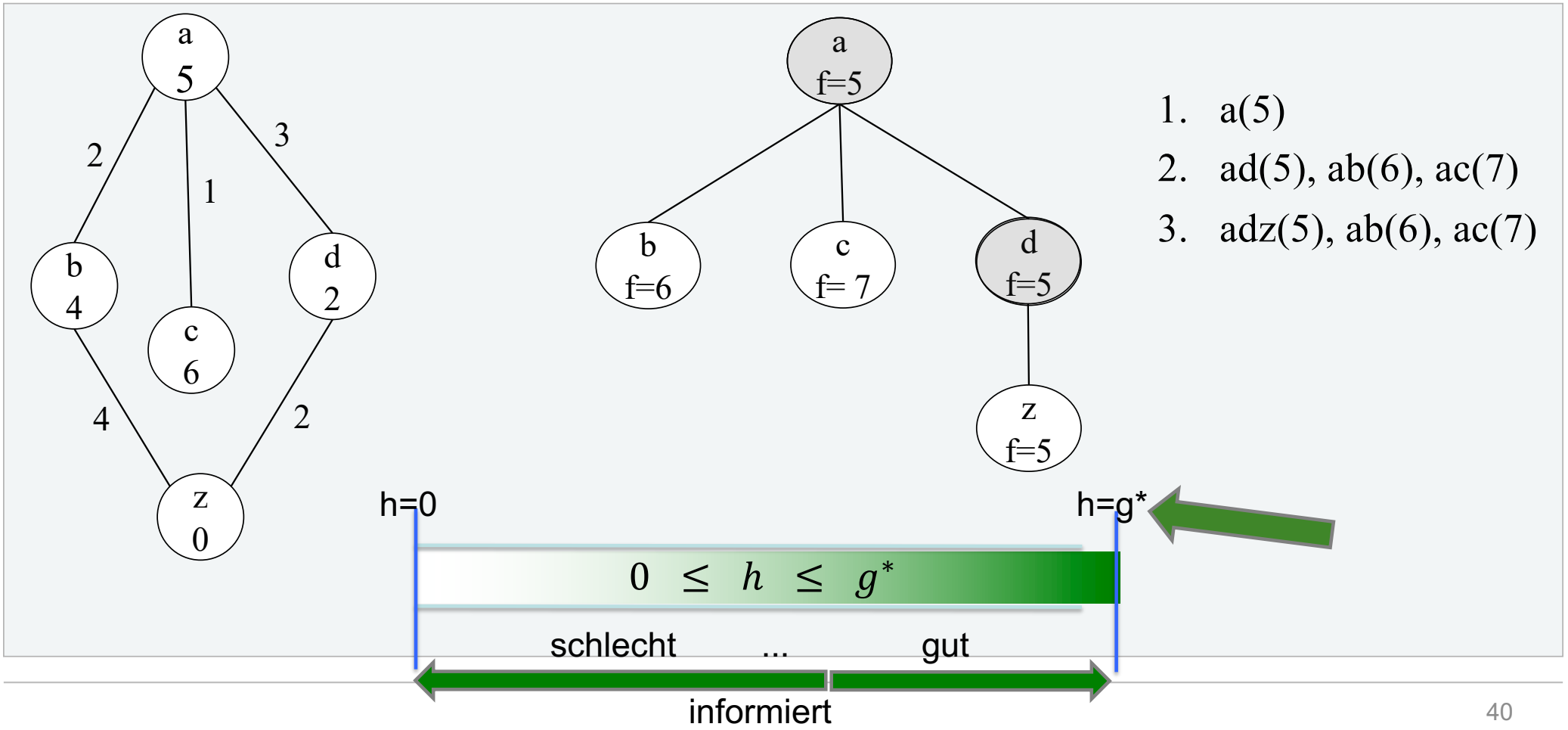
Mit nicht zulässiger Heuristik kann A* Optimalität nicht garantieren



Mit zulässiger Heuristik garantiert A* Optimalität

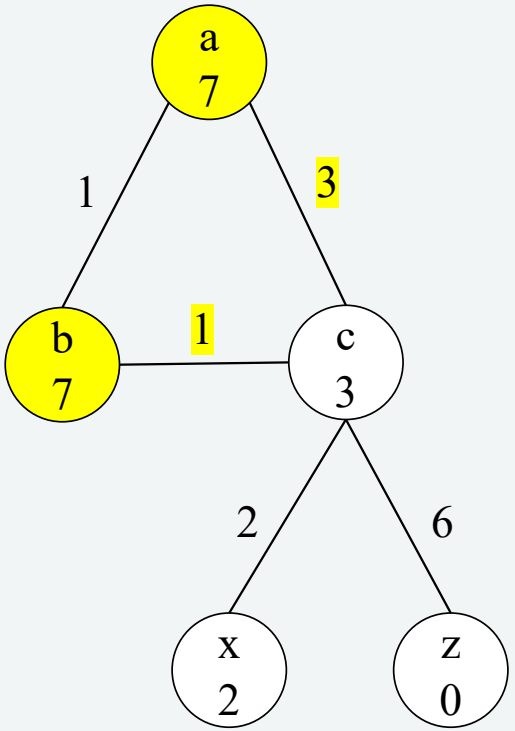


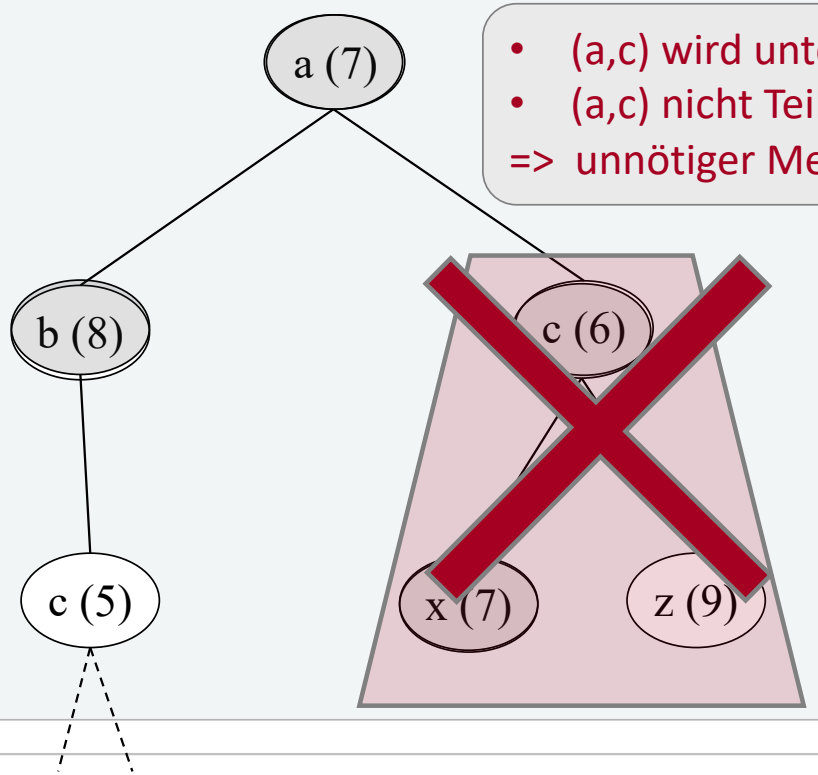
Mit optimaler Heuristik hat A* minimalen Aufwand



A* mit zulässiger aber nicht monotoner Heuristik

Monotonie: $\forall i\colon\ h(x_i) \leq h(x_{i+1}) + c(x_i, x_{i+1})$



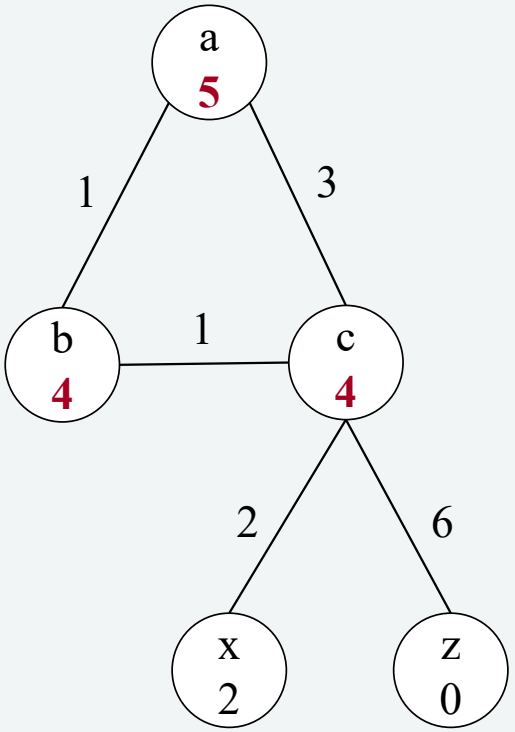


- (a,c) wird untersucht
- (a,c) nicht Teil der optimalen Lösung

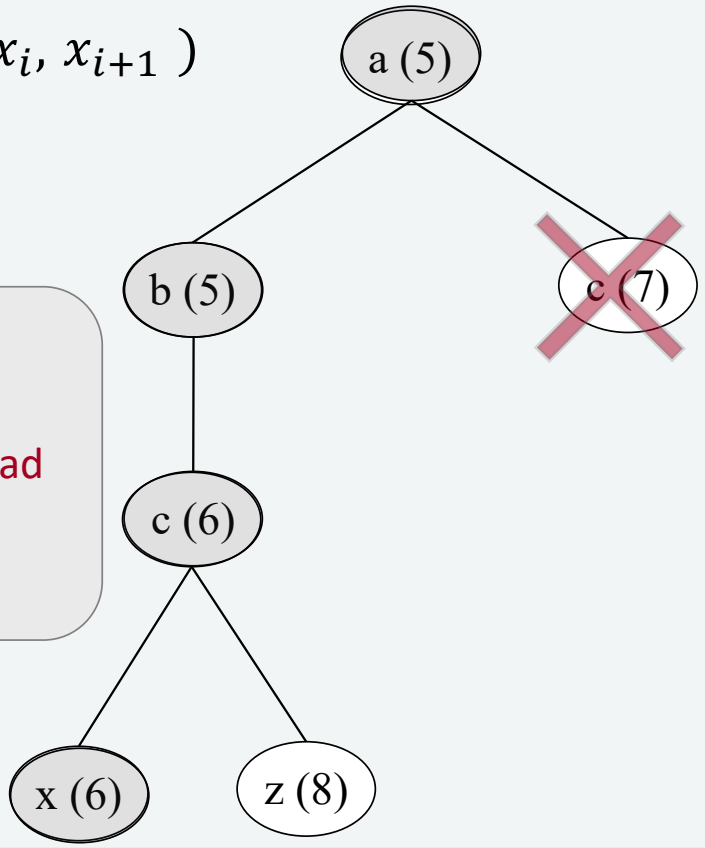
=> unnötiger Mehraufwand

Mit monotoner Heuristik besucht A* jeden Knoten auf dem kürzesten Pfad

Monotonie: $\forall i\colon\ h(x_i) \leq h(x_{i+1}) + c(x_i, x_{i+1})$



- monotone Heuristik: wachsende f-Werte
- jeder Zustand wird auf kürzestem Pfad besucht
- unnötiger Mehraufwand entfällt



A* ist ein guter Algorithmus (mit schlechten Worst-Case-Charakteristika)

A* ist „optimal effizient“[1]: A* expandiert:

- optimistisch: Knoten mit minimalem $f(n)$
- *alle* Knoten mit $f(n) \leq g^*$
- *keine* Knoten mit $f(n) > g^*$ (dynamische Programmierung)

Mit monotoner Heuristik: Worst-Case-Laufzeit quadratisch zur Zahl der Zustände.

- Hauptproblem: Speicherverbrauch
  → Simplified memory-bounded A* [Russell, 1992]

Vollständigkeit	Ja*
Optimalität	Ja*
Zeitkomplexität	$O(b^m)$
Speicherkomplexität	$O(b^m)$

b: Verzweigungsgrad

m: maximale Suchbaumtiefe

[1] kein anderer optimaler Algorithmus expandiert weniger Zustände

* mit zulässiger Heuristik und nichtnegativen Aktionskosten

Ausblick

- Speicherproblematik lösen
- Lokale Suche

© Dr.-Ing. Stefan Fricke

Auswege aus dem Speicherdilemma

Breitensuche-Verfahren haben hohen Speicherverbrauch:

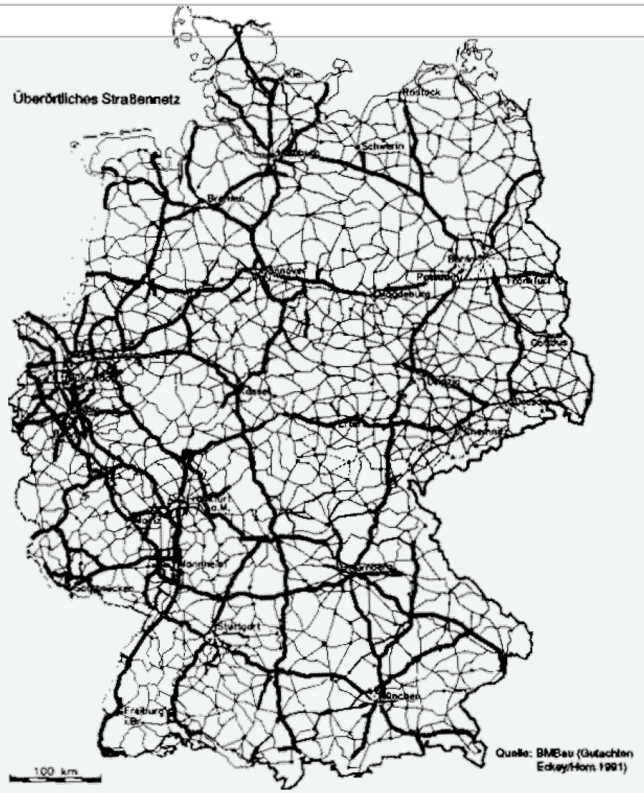
Zustand = 0,1 KB, Speicher = 1 GB => 10^7 Zustände

b=3 => Tiefe auf 15 limitiert

Lösungsansätze:

1. Iterative-deepening-A* [Korf 1985]
   angepasste Iterative Tiefensuche mit Kostenlimit f=g+h,
   initiale Kostengrenze = $h(s_0)$

2. Simplified memory-bounded A* [Russell, 1992]
   Wenn Speicher voll, teuerste Pfade temporär entfernen



R. Korf: Depth-first Iterative-Deepening: An Optimal Admissible Tree Search. Artificial Intelligence 27: 97–109, 1985.
S. Russell: Efficient memory-bounded search methods. Proceedings of the 10th European Conference on AI: 1–5, 1992.

Wenn der Weg egal ist, ist lokale Suche eine Option

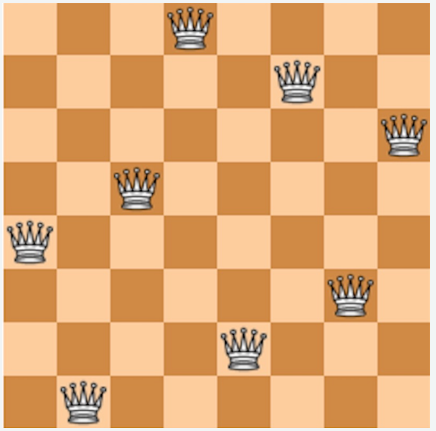
Iterative Verbesserung eines Zustands,
bis lokales Optimum erreicht



Job Shop Scheduling

1		2						
		3						
								4
	4			5				
	6			7				
							2	
	8							
						8		

Sudoku



Damenproblem

→ Lokale Suche wird beim Thema Constraint Satisfaction Problems behandelt.

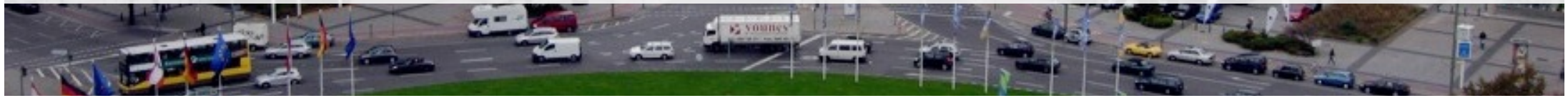


Dr.-Ing. Stefan Fricke

TU Berlin | Fachgebiet *Service-centric Networking*
Ernst-Reuter-Platz 1 | 10587 Berlin | Germany

stefan.fricke@tu-berlin.de

https://twitter.com/kuepp
https://www.xing.com/in/axelkuepper/
http://www.snet.tu-berlin.de/kuepper



Anhänge

Simplified memory-bounded A* (SMA*)

Breitensuche- und Tiefensuchesimulationen

© Dr.-Ing. Stefan Fricke

2.10 Speicheroptimierte Suche

- Simplified memory-bounded A* (SMA*) [Russell, 1992]

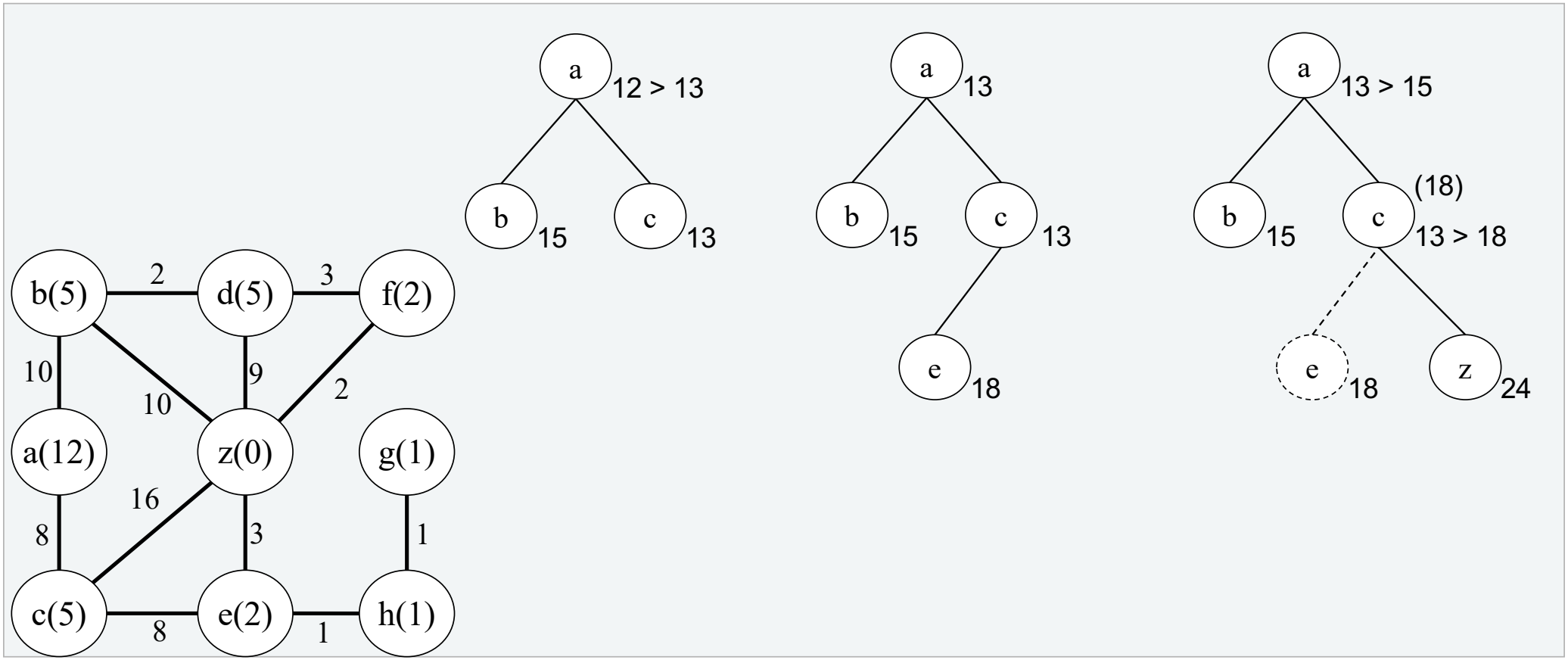
S. Russell: Efficient memory-bounded search methods. In Neumann, B. (ed.). Proceedings of the 10th European Conference on Artificial intelligence, 1992, pp. 1–5.

© Dr.-Ing. Stefan Fricke

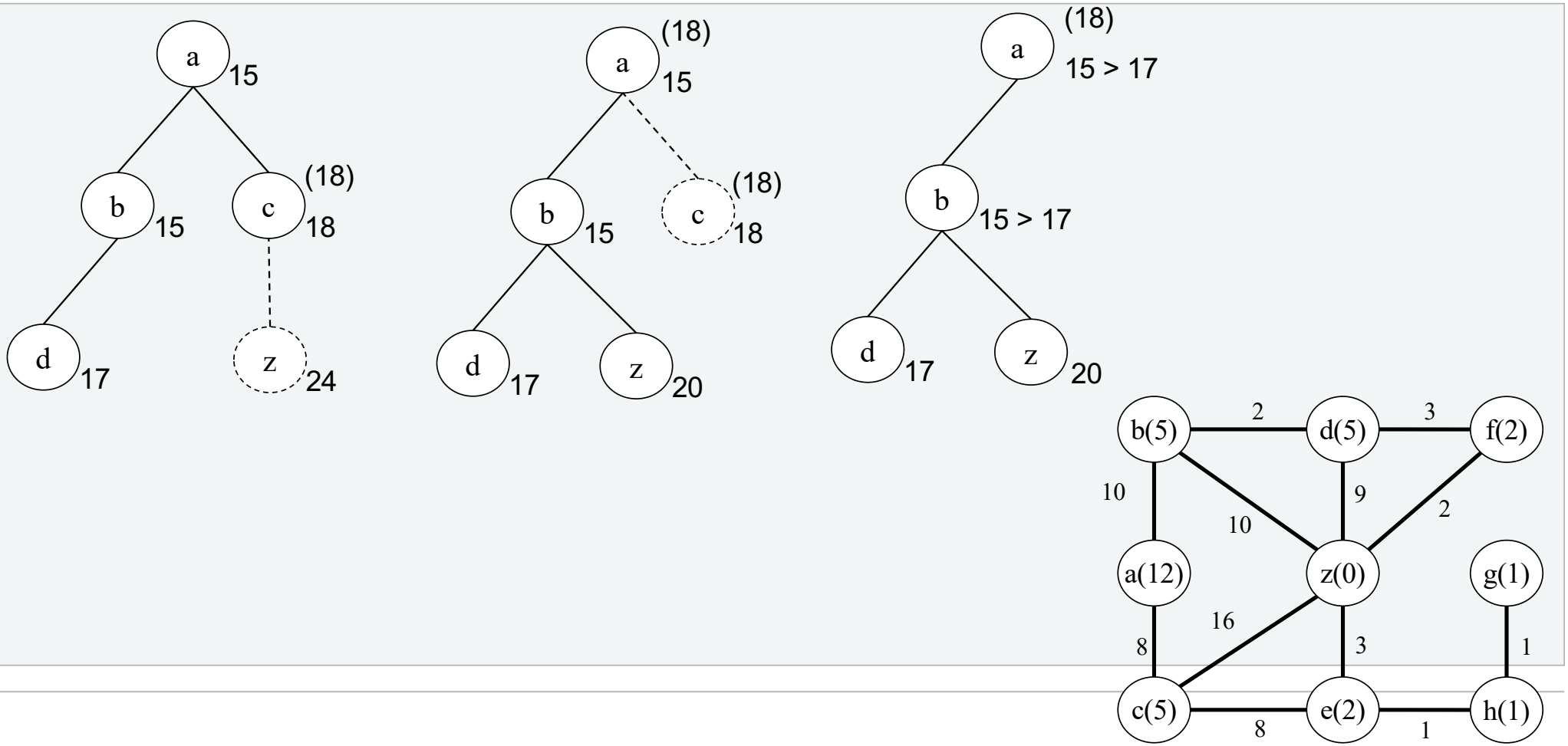
Simplified Memory-bounded A* (SMA*) nutzt begrenzten Speicher optimal

```
1  function SMA*(problem) returns a solution sequence
2    Queue ← MAKE-QUEUE({MAKE-NODE(INITIAL-STATE[problem])})
3    loop do
4         if Queue is empty then return failure
5         n ← deepest least-f-cost node in Queue
6         if GOAL-TEST(n) then return success
7         s ← NEXT-SUCCESSOR(n)
8         if s is not a goal and is at maximum depth then  f(s) ← ∞
9         else   f(s) ← MAX(f(n),g(s)+h(s))
10        if all of n's successors have been generated then
11           update n's f-cost and those of its ancestors if necessary
12        if SUCCESSORS(n) all in memory then remove n from Queue
13        if memory is full then
14           delete shallowest, highest-f-cost node in Queue
15           remove it from its parent's successor list
16           insert its parent on Queue if necessary
17        insert s in Queue
18    end
```

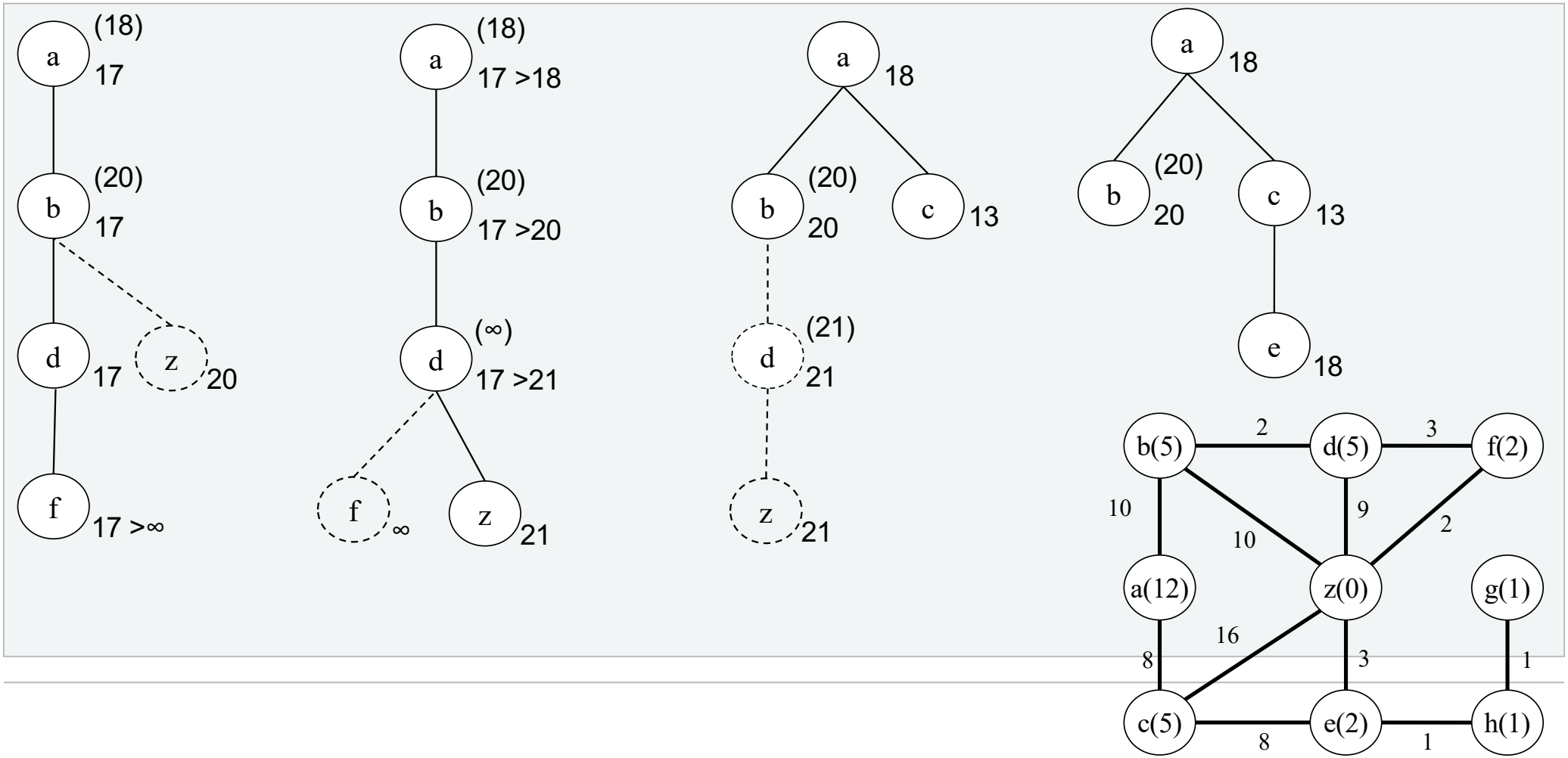
SMA*-Simulation mit einem Speicherlimit von 4 Zuständen (1/4)



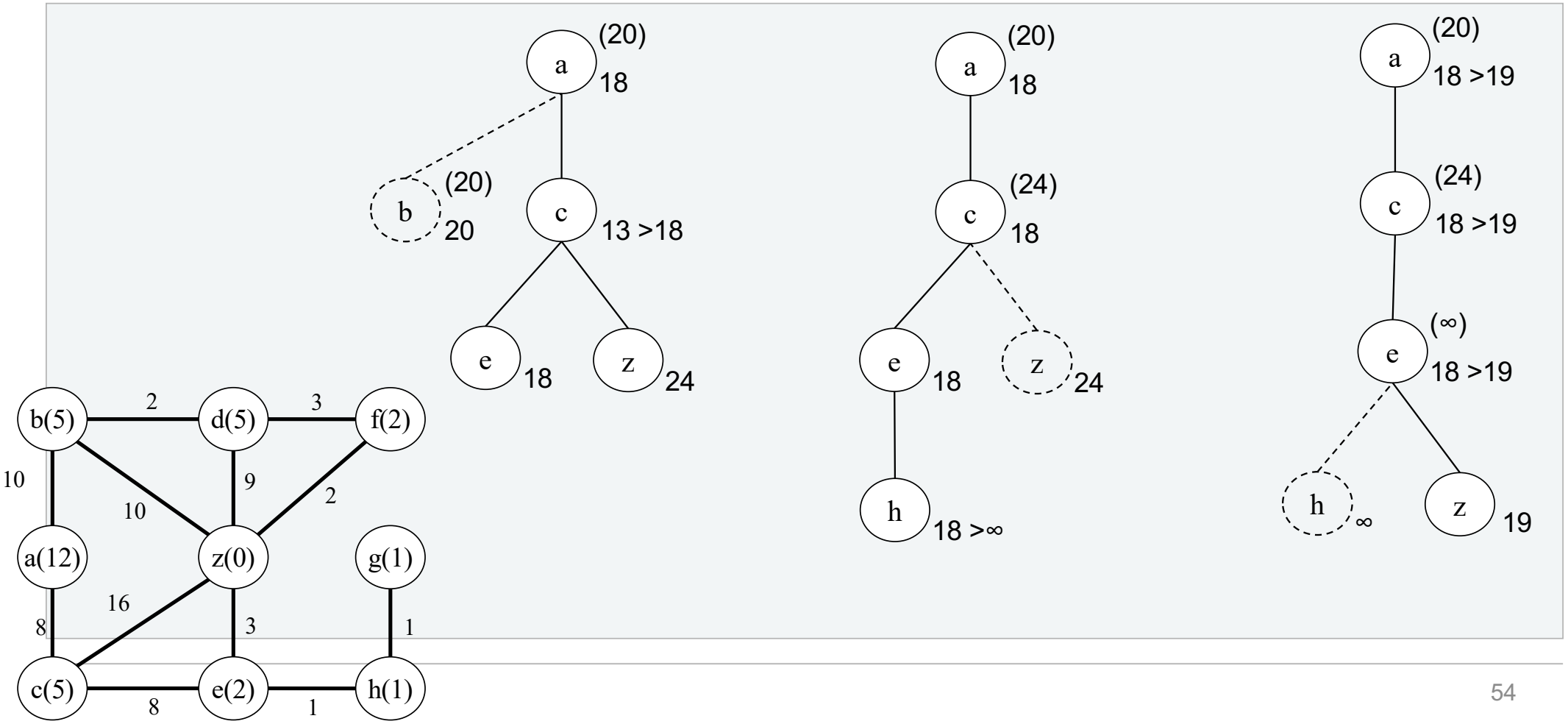
SMA*-Simulation (2/4)



SMA*-Simulation (3/4)



SMA*-Simulation (4/4)



Fazit SMA*

vollständig, wenn hinreichend Speicher für Lösung auf minimaler Baumtiefe

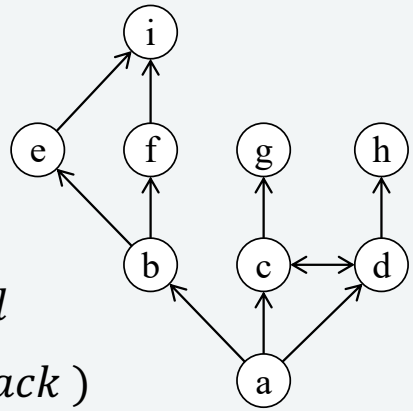
optimal, wenn hinreichend Speicher optimale Lösung (minimale Pfadkosten)

optimal effizient, wenn gesamter Suchbaum in Speicher passt

aufwändig durch Rekonstruktion „vergessener“ Knoten

Tiefensuchesimulation

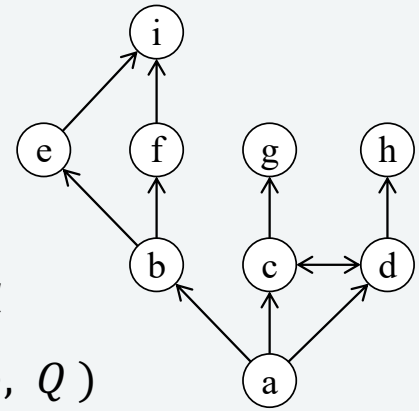
1 PUSH(s_0, $Stack$), $Generated = \{s_0\}$
2 Until $Stack$ nicht leer
3 $(s_0, \dots, s_k)$ = POP($Stack$)
4 IF $zt(s_k)$ THEN $return$ $(s_0, \dots, s_k)$
5 ELSE
6 NF = expandiere(s_k)
7 $\forall x \in NF$ mit $x \notin Generated$
8 PUSH($(s_0, \dots, s_k, x)$, $Stack$)
9 $Generated = Generated \cup NF$
10 Return Failure



Entwicklung des Stacks
1. a
2. ab, ac, ad
3. abe, abf, ac, ad
4. abei, abf, ac, ad
5. abf, ac, ad // i ∈ $Generated$
6. ac, ad
7. acg, ad // d ∈ $Generated$
8. ad // c ∈ $Generated$
9. adh
10. // Stack leer

Breitensuchesimulation

1 ENQUEUE(s_0, Q), $Generated = \{s_0\}$
2 Until Q nicht leer
3 $(s_0, \dots, s_k)$ = DEQUEUE(Q)
4 IF $zt(s_k)$ THEN $return$ $(s_0, \dots, s_k)$
5 ELSE
6 NF = expandiere(s_k)
7 $\forall x \in NF$ mit $x \notin Generated$
8 ENQUEUE($(s_0, \dots, s_k, x)$, Q)
9 $Generated = Generated \cup NF$
10 Return Failure



Entwicklung der Queue

1. a
2. ab, ac, ad
3. ac, ad, abe, abf
4. ad, abe, abf, acg // d redundant
5. abe, abf, acg, adh // c redundant
6. abf, acg, adh, abei
7. acg, adh, abei // i redundant
8. adh, abei
9. abei
10. // Q leer

A|O|T
Agententechnologien in betrieblichen Anwendungen und der Telekommunikation

Anhänge (nicht verwendete Folien)

Tipps zur Suchproblemrepräsentation

Simulationen

© Dr.-Ing. Stefan Fricke